Placement Portal Project Prompts

STEP 1

Hey AI, I am building the backend backend for an AI-powered placement portal using Python and FastAPI. I am not an expert coder, so please provide clean, structured, and production-ready code.

Requirements for Step 1:
1. Framework: Create a basic FastAPI application structure.
2. Gemini API Rotation Logic: I want to use a list of 3 different Google Gemini API Keys (Gemini 2.5 Flash model) to bypass daily free tier limits.
3. Rotation Implementation: Implement a safe Round-Robin logic. Every time an API request is made, the system should automatically pick the next key in the list. If one key hits a rate limit or returns an error, it must automatically retry the request using the next available key without crashing.
4. Test Endpoint: Create a simple POST endpoint `/api/test-gemini` that takes a basic prompt string, sends it to Gemini using the rotated key, and returns the text response.

Please give me the complete `main.py` code, the list of python libraries I need to install via pip, and the command to run this server locally.

STEP 2

Hey AI, now let's add Step 2 to our FastAPI backend. We need to create a standalone "ATS Resume Checker" feature. This is completely separate from the interview system.

Requirements for Step 2:
1. Endpoint: Create a POST endpoint `/api/ats/check-resume`. It must accept a PDF file (the resume) and a text string (the Job Description).
2. Privacy/In-Memory Parsing: Use a library like `PyPDF2` or `pdfplumber` to extract all text from the uploaded PDF directly in memory (RAM). Do NOT save the file anywhere on the hard disk or database.
3. Local NLP Scoring Logic (No Gemini Cost): Write a Python function using `scikit-learn` (TF-IDF Vectorizer and Cosine Similarity) or keyword-matching algorithms to calculate an ATS match score between the resume text and the job description.
4. Response: Return a structured JSON response containing:
- ats_score
- keywords_matched
- keywords_missing
- improvement_suggestions

Please update our existing FastAPI code to include this new endpoint and tell me what new python packages I need to install.

STEP 3

Hey AI, let's implement Step 3: The Standalone AI Mock Interview service.

Requirements:
1. Create `/api/interview/chat-round`.
2. Gemini asks exactly 3 technical questions at once.
3. Use the provided system prompt.
4. Accept previous chat history.
5. Return evaluation and next 3 questions in structured JSON.

Integrate with the active API key rotation system.

STEP 4

Hey AI, our Python FastAPI backend is working perfectly with Gemini rotation, standalone ATS parsing, and standalone batch interview endpoints.

Requirements:
1. Use React.js or Next.js with Tailwind CSS.
2. Create two tabs:
- ATS Resume Checker
- AI Mock Interview
3. Connect frontend to:
- `/api/ats/check-resume`
- `/api/interview/chat-round`

Provide the complete front-end code layout and instructions to run it.